

Documentação da Rota de Clientes

Visão geral

A rota de **clientes** centraliza as operações de cadastro e manutenção da base de clientes da ótica.

Ela permite:

- listar todos os clientes da conta logada;
- cadastrar novos clientes;
- editar clientes existentes;
- excluir clientes, somente quando o usuário autenticado for **administrador**.

Todas as operações são protegidas por autenticação e vinculadas à `conta_id` do usuário logado.

O front **não controla nem envia a conta válida**, essa associação é feita com segurança no backend.

Resumo da rota

Rota	Método	Finalidade
<code>/api/clientes</code>	GET	Listar todos os clientes da conta
<code>/api/clientes</code>	POST	Cadastrar um novo cliente
<code>/api/clientes</code>	PUT	Atualizar um cliente existente

<code>/api/clientes</code>	<code>DELETE</code>	Excluir um cliente, apenas admin
----------------------------	---------------------	----------------------------------

Regras gerais da rota

Autenticação

Antes de qualquer operação, a rota:

1. consulta o usuário autenticado com:

```
supabase.auth.getUser()
```

2. busca o perfil correspondente na tabela:

```
usuarios
```

3. valida se:

- o usuário existe;
 - o perfil foi encontrado;
 - o status do usuário é **ativo**.
-

Vínculo por conta

Toda operação usa:

```
profile.conta_id
```

Isso garante que:

- o usuário só veja clientes da própria conta;
 - o cliente criado seja salvo na conta correta;
 - não seja possível editar ou excluir cliente de outra ótica.
-

Estratégia de dados

A rota foi desenhada para ser simples e escalável:

- `GET` utiliza `select( "*")`;
- `POST` recebe o body completo;
- `PUT` recebe o body completo;
- os filtros da tela são feitos diretamente no front;
- a rota não fica presa a uma lista manual de campos espalhada em vários arquivos.

Essa escolha reduz manutenção e facilita adicionar novos campos futuramente.

1. Listar clientes

`GET /api/clientes`

Retorna todos os clientes pertencentes à conta do usuário autenticado.

Comportamento

A rota executa:

```
supabase
  .from("clientes")
  .select("*")
  .eq("conta_id", profile.conta_id)
  .order("created_at", { ascending: false });
```

Resposta de sucesso

200 OK

```
{
  "success": true,
  "clientes": [
```

```
{  
  "id": "uuid-do-cliente",  
  "conta_id": "uuid-da-conta",  
  "nome_completo": "Helena Duarte",  
  "nome_social": null,  
  "cpf": "123.456.789-10",  
  "rg": null,  
  "data_nascimento": "1998-04-12",  
  "telefone_principal": "(32) 99931-1012",  
  "telefone_secundario": null,  
  "email": "helena@email.com",  
  "cep": "36300-000",  
  "rua": "Rua das Flores",  
  "numero": "120",  
  "complemento": null,  
  "bairro": "Centro",  
  "cidade": "São João del-Rei",  
  "estado": "MG",  
  "pais": "Brasil",  
  "origem_cliente": "Instagram",  
  "profissao": "Professora",  
  "prefere_contato_por": "whatsapp",  
  "observacoes": null,  
  "ativo": true,  
  "created_at": "2026-05-18T13:20:00.000Z",  
  "updated_at": "2026-05-18T13:20:00.000Z"  
}
```

```
]
}
```

Erros possíveis

401 Unauthorized

Usuário sem sessão ativa.

```
{
  "error": "Usuário não autenticado."
}
```

404 Not Found

Perfil não encontrado na tabela `usuarios`.

```
{
  "error": "Perfil de usuário não encontrado."
}
```

403 Forbidden

Usuário com acesso inativo ou bloqueado.

```
{
  "error": "Seu acesso está inativo ou bloqueado."
}
```

400 Bad Request

Falha ao consultar a tabela de clientes.

```
{
  "error": "Não foi possível buscar os clientes."
}
```

500 Internal Server Error

Erro inesperado.

```
{  
  "error": "Erro interno ao buscar clientes."  
}
```

2. Cadastrar cliente

POST /api/clientes

Cria um novo cliente na conta do usuário autenticado.

Body

A rota recebe o body completo do formulário.

```
{  
  "nome_completo": "Helena Duarte",  
  "nome_social": "",  
  "cpf": "123.456.789-10",  
  "rg": "",  
  "data_nascimento": "1998-04-12",  
  "telefone_principal": "(32) 99931-1012",  
  "telefone_secundario": "",  
  "email": "helena@email.com",  
  "cep": "36300-000",  
  "rua": "Rua das Flores",  
  "numero": "120",  
  "complemento": "",  
  "bairro": "Centro",
```

```
"cidade": "São João del-Rei",  
"estado": "MG",  
"pais": "Brasil",  
"origem_cliente": "Instagram",  
"profissao": "Professora",  
"prefere_contato_por": "whatsapp",  
"observacoes": "Cliente prefere contato pela manhã.",  
"ativo": true  
}
```

Campos obrigatórios

A rota exige:

- `nome_completo`;
 - `telefone_principal`.
-

Validação de CPF duplicado

Quando o CPF for informado, a rota verifica se já existe outro cliente com o mesmo CPF dentro da mesma conta.

Consulta conceitual:

```
clientes  
  
where conta_id = conta_atual  
  
and cpf = cpf_informado
```

Normalização de dados

Antes de salvar, a rota:

- remove espaços extras;

- converte strings vazias em `null`;
- ignora campos que não devem ser controlados pelo front:
 - `id`;
 - `conta_id`;
 - `created_at`;
 - `updated_at`.

O campo `conta_id` é inserido exclusivamente pelo backend.

Resposta de sucesso

201 Created

```
{  
  "success": true,  
  "message": "Cliente cadastrado com sucesso.",  
  "cliente": {  
    "id": "uuid-do-cliente",  
    "conta_id": "uuid-da-conta",  
    "nome_completo": "Helena Duarte",  
    "telefone_principal": "(32) 99931-1012",  
    "ativo": true  
  }  
}
```

Erros possíveis

400 Bad Request

Nome não informado.

```
{  
  "error": "Informe o nome completo do cliente."
```

```
}
```

Telefone não informado.

```
{
```

```
"error": "Informe o telefone principal do cliente."
```

```
}
```

Falha ao inserir.

```
{
```

```
"error": "Não foi possível cadastrar o cliente."
```

```
}
```

409 Conflict

CPF já cadastrado na mesma conta.

```
{
```

```
"error": "Já existe um cliente cadastrado com este CPF."
```

```
}
```

401 Unauthorized

```
{
```

```
"error": "Usuário não autenticado."
```

```
}
```

404 Not Found

```
{
```

```
"error": "Perfil de usuário não encontrado."
```

```
}
```

403 Forbidden

```
{
```

```
"error": "Seu acesso está inativo ou bloqueado."
```

```
}
```

500 Internal Server Error

```
{  
  "error": "Erro interno ao cadastrar cliente."  
}
```

3. Atualizar cliente

PUT /api/clientes

Atualiza os dados de um cliente existente.

Body

A atualização recebe o body completo, incluindo o **id**.

```
{  
  "id": "uuid-do-cliente",  
  "nome_completo": "Helena Duarte de Oliveira",  
  "nome_social": "",  
  "cpf": "123.456.789-10",  
  "rg": "",  
  "data_nascimento": "1998-04-12",  
  "telefone_principal": "(32) 99931-1012",  
  "telefone_secundario": "",  
  "email": "helena@email.com",  
  "cep": "36300-000",  
  "rua": "Rua das Flores",  
  "numero": "120",
```

```
"complemento": "Casa",  
"bairro": "Centro",  
"cidade": "São João del-Rei",  
"estado": "MG",  
"pais": "Brasil",  
"origem_cliente": "Instagram",  
"profissao": "Professora",  
"prefere_contato_por": "whatsapp",  
"observacoes": "Atualização cadastral realizada.",  
"ativo": true  
}
```

Validações

A rota exige:

- `id` do cliente;
- `nome_completo`;
- `telefone_principal`.

Também verifica:

- se o cliente pertence à conta do usuário logado;
 - se o CPF informado não pertence a outro cliente da mesma conta.
-

Comportamento

1. valida autenticação e perfil;
2. valida campos obrigatórios;
3. busca o cliente por:
 - `id`;
 - `conta_id`;
4. valida duplicidade de CPF;
5. atualiza os dados;
6. redefine:

updated_at: new Date().toISOString()

Resposta de sucesso

200 OK

```
{
  "success": true,
  "message": "Cliente atualizado com sucesso.",
  "cliente": {
    "id": "uuid-do-cliente",
    "nome_completo": "Helena Duarte de Oliveira",
    "telefone_principal": "(32) 99931-1012",
    "updated_at": "2026-05-18T14:55:00.000Z"
  }
}
```

Erros possíveis

400 Bad Request

ID ausente.

```
{
  "error": "ID do cliente não informado."
}
```

Nome ausente.

```
{
  "error": "Informe o nome completo do cliente."
}
```

Telefone ausente.

```
{  
  "error": "Informe o telefone principal do cliente."  
}
```

Falha ao atualizar.

```
{  
  "error": "Não foi possível atualizar o cliente."  
}
```

404 Not Found

Cliente não encontrado ou fora da conta do usuário.

```
{  
  "error": "Cliente não encontrado."  
}
```

409 Conflict

CPF pertence a outro cliente.

```
{  
  "error": "Já existe outro cliente cadastrado com este CPF."  
}
```

401 Unauthorized

```
{  
  "error": "Usuário não autenticado."  
}
```

403 Forbidden

```
{  
  "error": "Seu acesso está inativo ou bloqueado."  
}
```

500 Internal Server Error

```
{  
  "error": "Erro interno ao atualizar cliente."  
}
```

4. Excluir cliente

DELETE /api/clientes

Exclui um cliente da conta.

Essa ação é permitida **somente para usuários com role = "admin"**.

Body

```
{  
  "id": "uuid-do-cliente"  
}
```

Permissão

Antes de excluir, a rota valida:

```
profile.role === "admin"
```

Caso contrário, a exclusão é bloqueada.

Comportamento

1. valida autenticação;
2. valida se o perfil está ativo;
3. valida se o usuário é admin;
4. valida se o `id` foi enviado;

5. busca o cliente dentro da conta atual;
6. executa exclusão;
7. devolve confirmação.

Resposta de sucesso

200 OK

```
{  
  "success": true,  
  "message": "Cliente excluído com sucesso."  
}
```

Erros possíveis

403 Forbidden

Usuário sem permissão de exclusão.

```
{  
  "error": "Apenas administradores podem excluir clientes."  
}
```

400 Bad Request

ID ausente.

```
{  
  "error": "ID do cliente não informado."  
}
```

Falha genérica ao excluir.

```
{  
  "error": "Não foi possível excluir o cliente."  
}
```

404 Not Found

Cliente não localizado na conta atual.

```
{  
  "error": "Cliente não encontrado."  
}
```

409 Conflict

Cliente possui vínculos com outras tabelas e não pode ser removido.

```
{  
  "error": "Este cliente possui registros vinculados e não pode ser excluído."  
}
```

401 Unauthorized

```
{  
  "error": "Usuário não autenticado."  
}
```

500 Internal Server Error

```
{  
  "error": "Erro interno ao excluir cliente."  
}
```

Padrão de segurança adotado

A rota de clientes segue quatro proteções principais:

1. Sessão obrigatória

Nenhuma operação funciona sem usuário autenticado.

2. Perfil ativo

Usuários inativos ou bloqueados não manipulam dados.

3. Isolamento por conta

Toda leitura, edição e exclusão usa:

```
.eq("conta_id", profile.conta_id)
```

4. Exclusão restrita

Apenas `admin` pode excluir cliente.

-